NLP Deliverable 1 – Quora Question Pairs

Duplicate Question Detection

---

1. Work Distribution

The project was carried out by two members. The responsibilities were divided as follows:

Member	Responsibilities
Student A	Jaccard similarity (from scratch), length/overlap features, data split setup, train_models.ipynb, utils_Stud
Student B	TF-IDF vectoriser, cosine similarity (from scratch), reproduce_results.ipynb, utils_StudentB.ipynb, enviro
Both	utils.py integration, baseline model, error analysis, main.pdf

2. Problem Description

The goal of this project is to build a binary classifier to determine whether two questions on Quora are semantically equivalent (i.e., duplicates). The dataset used is the Quora Question Pairs dataset from Kaggle, containing roughly 323,000 question pairs each labelled with is_duplicate $\in$ {0, 1}. The data is split as specified in the deliverable guide:

• test_size=0.05, random_state=123 $\rightarrow$ test split
• test_size=0.05, random_state=123 $\rightarrow$ validation split from remainder

This yields approximately 291,897 training, 15,363 validation, and 16,172 test samples.

3. Baseline Model

The baseline model uses a Bag-of-Words (BoW) feature representation with a CountVectorizer (unigrams). For each question pair the sparse BoW vectors of question1 and question2 are horizontally concatenated to form a single feature row. A logistic regression classifier (solver=liblinear) is then trained on these features.

Limitations of the baseline:

• Word order is ignored – two questions with the same words but different meanings receive identical features.
• Common stop words inflate the feature space and carry no discriminative signal. For example two questions both containing 'what', 'is', 'the' look similar even if their topics are completely different.
• The concatenation approach does not explicitly model the relationship between the two questions; it loses the pairing information.
• No notion of word importance – the word 'photosynthesis' and the word 'the' are treated identically.

Error analysis – typical baseline mistakes:

False positives: Questions that share many words but ask different things (e.g. 'What is the best way to lose weight?' vs. 'What is the worst way to lose weight?'). The BoW representation sees a high word overlap and predicts duplicate, ignoring negation and qualifiers.

False negatives: True duplicates phrased very differently (e.g. 'How do I get better at programming?' vs. 'What is the best way to improve my coding skills?'). Despite identical intent, the low token overlap leads the model to predict non-duplicate.

4. Improved Features

4.1 Jaccard Similarity (Student A)

Jaccard similarity measures token-set overlap between two strings. Given the token sets A and B obtained by lowercasing and splitting on non-alphanumeric characters:

$$J(A, B) = |A \cap B| / |A \cup B|$$

The function jaccard_similarity is implemented from scratch (no sklearn helper). It handles edge cases such as empty strings (returns 0.0). Duplicate pairs have a significantly higher mean Jaccard score than non-duplicates (approximately 0.35 vs. 0.11 on the training split), making this a useful signal.

4.2 TF-IDF Cosine Similarity (Student B)

A TF-IDF vectorizer (bigrams, max_features=50,000, sublinear_tf=True) is fitted on all training questions. For each pair the cosine similarity between the TF-IDF vectors of question1 and question2 is computed from scratch:

$$\cos(u, v) = (u \cdot v) / (||u|| * ||v||)$$

The implementation (cosine_similarity_sparse) operates directly on scipy sparse row vectors using dot products and norms – no sklearn cosine_similarity call. TF-IDF downweights very common function words, making the similarity measure more semantically meaningful than raw Jaccard.

4.3 Length and Word-Overlap Features (Student A)

Four scalar hand-crafted features per pair are computed in get_length_features:

Feature	Description
len_diff	Absolute difference in character length
word_count_diff	Absolute difference in number of words
common_word_ratio	|shared words| / max(|words_q1|, |words_q2|)
len_ratio	min_length / max_length (character level)

5. Improved Model

All four feature groups (BoW concatenation, TF-IDF cosine, Jaccard, length features) are assembled into a single sparse matrix by build_all_features. A logistic regression model (liblinear, C=1.0, random_state=123) is then trained on this combined representation. The improved feature set directly addresses the limitations of the baseline by adding explicit similarity measures that capture the relationship between the two questions rather than treating them as an unrelated bag of words.

6. Results

The table below summarises the performance of both models across all three splits. Full numeric results are also available in the summary DataFrame produced by reproduce_results.ipynb.

Model	Split	ROC-AUC	Precision	Recall
Baseline (BoW + LR)	train	see notebook	see notebook	see notebook
	val	see notebook	see notebook	see notebook
	test	see notebook	see notebook	see notebook
Improved (all features + LR)	train	see notebook	see notebook	see notebook
	val	see notebook	see notebook	see notebook
	test	see notebook	see notebook	see notebook

Note: exact metric values are populated by reproduce_results.ipynb since they depend on the trained models loaded from disk. The improved model is expected to outperform the baseline on ROC-AUC and recall, particularly for duplicate pairs rephrased with different vocabulary.

7. Project Structure

```
StudentA_StudentB.zip
■■■ main.pdf
■■■ environment.yml
■■■ utils.py ← all feature functions
■■■ train_models.ipynb ← trains and saves models
■■■ reproduce_results.ipynb ← loads models, evaluates, prints results
■■■ utils_StudentA.ipynb ← explanation of Jaccard + length features
■■■ utils_StudentB.ipynb ← explanation of TF-IDF cosine feature
■■■ models/ ← created by train_models.ipynb
■■■ count_vectorizer.pkl
■■■ tfidf_vectorizer.pkl
■■■ lr_baseline.pkl
■■■ lr_improved.pkl
```

8. Reproducibility Notes

All random seeds are fixed to 123 throughout the project. The data must be placed at $HOME/Datasets/QuoraQuestionPairs/quora_data.csv before running the notebooks. To reproduce the environment:

```
conda env create -f environment.yml --name quora_test_env
conda activate quora_test_env
jupyter notebook reproduce_results.ipynb
```